

Reviewer Pack — Arithmetic Geometry of Function Fields Bounds Resolution Pro...

Entry e12adacb1893 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 19:29:49 UTC

Arithmetic Geometry of Function Fields Bounds Resolution Proof Width

Entry ID: e12adacb1893 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 19:29:49 UTC

1. Conjecture

Field A (mathematical branch): Arithmetic Geometry (Function Fields) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

[‘For a given CNF formula F with n variables, the arithmetic genus $g(F)$ of its characteristic function field is upper bounded by the resolution proof width ω_F .’, ‘If there exists an instance F such that $g(F) > \omega_F$, then F has resolution proof width at least $10n$.’]

Rationale (proposer’s reasoning):

[‘Arithmetic geometry provides a rich framework for studying algebraic varieties. Function fields are a natural extension of this idea to study polynomials over finite fields.’, ‘Resolution proof complexity is a fundamental measure of the difficulty of solving SAT problems. By linking arithmetic geometry with resolution proofs, we may uncover new structural insights into the computational complexity of SAT.’]

Taxonomy category: SOS_DEGREE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3fcc06981e04154d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 80% of the 30 random seeds tested, the arithmetic genus $g(F)$ does not exceed the resolution proof width ω_F by more than a factor of 10 and the average correlation coefficient between $g(F)$ and ω_F across all seeds is greater than or equal to 0.7. The conjecture is falsified if any seed produces an arithmetic genus $g(F)$ greater than 10 times its corresponding resolution proof width ω_F .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	HITS	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - arithmetic geometry AND function fields AND complexity theory AND resolution proof width - resolution proof complexity AND upper bound ON arithmetic genus AND characteristic function fields - CNF formula AND arithmetic genus $g(F)$ AND resolution proof width $\omega_F \geq 10n$

Top relevant hits considered: - [<http://arxiv.org/abs/0706.3841v1>] Arithmetic lattices and weak spectral geometry - [<http://arxiv.org/abs/hep-th/0307066v2>] On Superconformal Field Theories Associated to Very Attractive Quartics - [<http://arxiv.org/abs/1705.05818v2>] Noether's Theorem in Multisymplectic Geometry - [<http://arxiv.org/abs/1109.2952v4>] Upper bound on distance in the pants complex - [<http://arxiv.org/abs/1606.05050v1>] Proof Complexity Lower Bounds from Algebraic Circuit Complexity - [<http://arxiv.org/abs/1706.04100v1>] Determination of spin and parity of the $Z_c(3900)$ - [<http://arxiv.org/abs/0803.2366v1>] Arithmetic Riemann-Roch and Hilbert-Samuel formulae for pointed stable curves - [<http://arxiv.org/abs/2112.11839v2>] Two Formulas for F -Polynomials

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(10 * n):
            clause = set(random.sample(range(1, n + 1), random.randint(1, n)))
            if random.choice([True, False]):
                clause = {x: -1 for x in clause}
            else:
                clause = {x: 1 for x in clause}
            clauses.append(clause)
        return clauses

    def resolution_proof_width(cnf):
```

```

clauses = cnf.copy()
while True:
    new_clauses = []
    added_clause = False
    for i in range(len(clauses)):
        for j in range(i + 1, len(clauses)):
            if -clauses[i].intersection(clauses[j]):
                new_clause = {x: -1 for x in clauses[i] if x not in clauses[j]}
                new_clause.update({x: -1 for x in clauses[j] if x not in clauses[i]})
                new_clauses.append(new_clause)
                added_clause = True
    if not added_clause:
        break
    clauses.extend(new_clauses)
return len(clauses)

def arithmetic_genus(cnf):
    n = len(cnf[0])
    genus = 0
    for clause in cnf:
        if any(x < 0 for x in clause.values()):
            genus += 1
    return genus

n = random.choice([5, 10, 15, 20, 30, 40])
cnf_formula = generate_cnf(n)

g_F = arithmetic_genus(cnf_formula)
w_F = resolution_proof_width(cnf_formula)

return {
    "metric_name": "arithmetic genus vs resolution proof width",
    "metric_value": abs(g_F - w_F),
    "instances_tested": 1,
    "conjecture_holds": g_F <= 10 * w_F,
    "counterexample": "" if g_F <= 10 * w_F else f"Arithmetic genus {g_F} > 10 * resolution proof width {w_F}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_metric = sum(r["metric_value"] for r in results) / len(results)
    std_metric = math.sqrt(sum((r["metric_value"] - mean_metric) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])

```

```

    print(f"RESULT: FALSIFIED counterexample=\"Arithmetic genus > 10 * resolution proof width\"
else:
    print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_65ecb1f8.py", line 77, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_65ecb1f8.py", line 61, in run_trial
    w_F = resolution_proof_width(cnf_formula)
          ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_65ecb1f8.py", line 39, in resolution_pr
    if -clauses[i].intersection(clauses[j]):
        ~~~~~
AttributeError: 'dict' object has no attribute 'intersection'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified according to the pre-registered criteria. | next: Investigate and fix the crash in the test code to allow for a proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11099
2	preregistration	ollama_remote	glm4:latest	0	0	5860
3	novelty	ollama_remote	glm4:latest	0	0	4716
4	novelty	ollama_remote	glm4:latest	0	0	7581
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20546
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14443
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11737
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10411
9	judge	ollama_remote	glm4:latest	0	0	10224

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 96617 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/e12adacb1893.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/e12adacb1893.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/e12adacb1893.tar.gz (if generated)